GCO Inference Demo Walkthrough

End-to-end demo of deploying, testing, scaling, and managing a GPU inference endpoint across multiple regions with GCO.

Table of Contents

Prerequisites

- GCO infrastructure deployed (`gco stacks deploy-all -y`)
- CLI installed (`pipx install -e .`)
- AWS credentials configured with access to the deployed account

Step 1: Verify Infrastructure

List the deployed stacks, then confirm the cluster is reachable:

```
gco stacks list
gco jobs health --all-regions
```

Step 2: Deploy a vLLM Inference Endpoint

Deploy a vLLM endpoint using the default `facebook/opt-125m` model (small, loads in seconds, good for testing). Omitting `-r` deploys to all regions so Global Accelerator routing works consistently.

```
gco inference deploy vllm-demo \
  -i vllm/vllm-openai:v0.22.0 \
  --gpu-count 1 \
  --replicas 1 \
  --extra-args '--model' --extra-args 'facebook/opt-125m'
```

For a larger model like Llama 3.1 8B, pass the model name and increase GPUs:

```
gco jobs submit-direct examples/inference-vllm.yaml -r us-east-1
```

You can also specify spot instances for cost savings on fault-tolerant workloads:

```
gco inference deploy vllm-spot \
  -i vllm/vllm-openai:v0.22.0 \
  --gpu-count 1 \
  --capacity-type spot \
  --extra-args '--model' --extra-args 'facebook/opt-125m'
```

Step 3: Monitor Deployment

The inference_monitor in each region picks up the DynamoDB record and creates the Kubernetes Deployment, Service, and Ingress on the shared ALB.

Watch status until all regions show “running”, then list every endpoint:

```
gco inference status vllm-demo
gco inference list
```

Wait for the status to show `running` in all target regions. The default opt-125m model typically takes 1-2 minutes. Larger models take longer depending on download speed and GPU scheduling.

Step 4: Invoke the Endpoint

Once the endpoint is running, send a prompt through the API Gateway:

```
gco inference invoke vllm-demo \
  -p "Explain GPU orchestration for ML workloads."
```

The command auto-detects that the image is vLLM and sends an OpenAI-compatible `/v1/completions` request via the API Gateway (SigV4 authenticated). The response text is printed directly.

Control output length:

```
gco inference invoke vllm-demo \
  -p "Write a haiku about Kubernetes." \
  --max-tokens 50
```

Send a raw JSON body for full control:

```
gco inference invoke vllm-demo \
  -d '{"model": "facebook/opt-125m", "prompt": "Hello!",
       "max_tokens": 30}'
```

Stream the response for lower time-to-first-token:

```
gco inference invoke vllm-demo \
  -p "Tell me about EKS Auto Mode." \
  --stream
```

Step 5: Scale the Endpoint

Scale to 3 replicas across all target regions, then confirm:

```
gco inference scale vllm-demo --replicas 3
gco inference status vllm-demo
```

Step 6: Enable Autoscaling

Deploy a second endpoint with HPA-based autoscaling:

```
gco inference deploy vllm-auto \
  -i vllm/vllm-openai:v0.22.0 \
  --gpu-count 1 \
  --replicas 2 \
  --min-replicas 1 --max-replicas 8 \
  --autoscale-metric cpu:70 \
  --extra-args '--model' --extra-args 'facebook/opt-125m'
```

The inference_monitor creates a Kubernetes HPA that scales between 1 and 8 replicas based on CPU utilization.

```
gco inference status vllm-auto
```

Step 7: Rolling Image Update

Update the image, then watch the rollout:

```
gco inference update-image vllm-demo -i vllm/vllm-openai:v0.22.0
gco inference status vllm-demo
```

Step 8: Canary Deployment

Test a new image version with a percentage of traffic before fully rolling out. Send 10% of traffic to the canary, then monitor both primary and canary:

```
gco inference canary vllm-demo \
  -i vllm/vllm-openai:v0.22.0 \
  --weight 10
gco inference status vllm-demo
```

If the canary looks good, promote it to primary (100% traffic):

```
gco inference promote vllm-demo -y
```

Or roll back if something is wrong:

```
gco inference rollback vllm-demo -y
```

Step 9: Stop and Restart

Stop the endpoint (scales to zero, keeps config in DynamoDB) and check status — it shows `stopped` with 0/0 replicas:

```
gco inference stop vllm-demo -y
gco inference status vllm-demo
```

Restart it and check status again — it shows `running` once the pods reschedule:

```
gco inference start vllm-demo
gco inference status vllm-demo
```

Step 10: Health Checks and Model Discovery

Check if an endpoint is healthy and ready to serve:

```
gco inference health vllm-demo
```

Discover which models are loaded on the endpoint:

```
gco inference models vllm-demo
```

This queries the OpenAI-compatible `/v1/models` path and returns the loaded model names, context lengths, and other metadata.

Step 11: Model Weight Management

Upload model weights to the central S3 bucket for use with inference endpoints across all regions.

Upload the weights, list the registered models, then get the S3 URI for use with `--model-source`:

```
gco models upload ./my-model-weights/ --name llama3-8b
gco models list
gco models uri llama3-8b
```

Deploy an endpoint with the model weights from S3, then clean up the registry entry when you are done:

```
gco inference deploy llama-endpoint \
  -i vllm/vllm-openai:v0.22.0 \
  --gpu-count 1 \
  --model-source $(gco models uri llama3-8b)
gco models delete llama3-8b -y
```

Step 12: Deploy on AWS Trainium or Inferentia

GCO supports AWS Trainium and Inferentia accelerators for cost-optimized inference. Use the `--accelerator neuron` flag to deploy on Neuron instances instead of NVIDIA GPUs.

Deploy on Inferentia (cost-optimized), check status, invoke it the same way as a GPU endpoint, then clean up:

```
gco inference deploy neuron-demo \
  -i public.ecr.aws/neuron/your-neuron-image:latest \
  --gpu-count 1 --accelerator neuron
gco inference status neuron-demo
gco inference invoke neuron-demo -p "What is machine learning?"
gco inference delete neuron-demo -y
```

The `--accelerator neuron` flag configures the deployment to:

- Request `aws.amazon.com/neuron` resources instead of `nvidia.com/gpu`
- Add the `aws.amazon.com/neuron` toleration for the Neuron nodepool
- Set node selectors for Neuron-capable instances

Container images must include the Neuron runtime — use images from `public.ecr.aws/neuron/` or build your own with the Neuron SDK.

Step 13: Valkey K/V Cache (Optional)

Each regional stack can include a Valkey Serverless cache for low-latency key-value storage. Use cases include prompt caching, session state, feature stores, and shared state across inference pods.

Enable Valkey in `cdk.json`:

```
"valkey": {
  "enabled": true,
  "max_data_storage_gb": 5,
  "max_ecpu_per_second": 5000,
  "snapshot_retention_limit": 1
}
```

Then redeploy the regional stack:

```
gco stacks deploy gco-us-east-1 -y
```

When Valkey is enabled, GCO automatically creates a `gco-valkey` ConfigMap in each namespace with the endpoint and port. Pods reference it via `configMapKeyRef` — no hardcoded URLs, and the same manifest works in any region:

```
env:
- name: VALKEY_ENDPOINT
  valueFrom:
    configMapKeyRef:
      name: gco-valkey
      key: endpoint
- name: VALKEY_PORT
  valueFrom:
    configMapKeyRef:
      name: gco-valkey
      key: port
```

Run the example job:

```
kubectl apply -f examples/valkey-cache-job.yaml
kubectl logs job/valkey-cache-example -n gco-jobs
```

The endpoint is also available via SSM at `/{project}/valkey-endpoint-{region}` for use outside the cluster (scripts, Lambda functions, etc.).

RAG with Semantic Caching

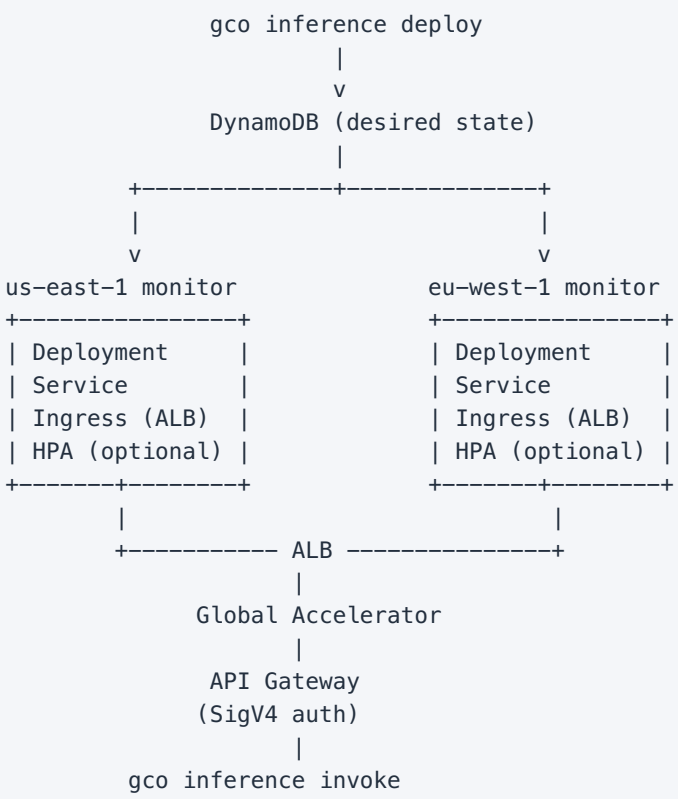
Valkey is ideal for semantic caching in RAG workflows — cache inference results keyed by prompt hash to avoid redundant GPU calls. For the vector store component, use Amazon OpenSearch Serverless, Bedrock Knowledge Bases, or pgvector. See Inference Guide — RAG Patterns for architecture details and code examples.

Step 14: Clean Up

Delete the demo endpoints, then verify they are gone:

```
gco inference delete vllm-demo -y
gco inference delete vllm-auto -y
gco inference list
```

Architecture



All inference endpoints share the main ALB via EKS Auto Mode's IngressClassParams `group.name`. Pods serve at the `/inference/{name}` prefix natively via `--root-path`.

The inference_monitor continuously reconciles desired state (DynamoDB) with actual state (Kubernetes). If a Deployment, Service, or Ingress is deleted or modified, the monitor recreates it automatically (self-healing).

Quick Reference

Command	Description
`gco inference deploy NAME -i IMAGE`	Deploy endpoint
`gco inference deploy NAME -i IMAGE --capacity-type spot`	Deploy on spot instances
`gco inference list`	List all endpoints
`gco inference status NAME`	Per-region status
`gco inference invoke NAME -p "..."`	Send a prompt
`gco inference invoke NAME -p "..." --stream`	Stream response
`gco inference scale NAME --replicas N`	Set replica count
`gco inference update-image NAME -i IMG`	Rolling update
`gco inference canary NAME -i IMG --weight 10`	Start canary deployment
`gco inference promote NAME -y`	Promote canary to primary
`gco inference rollback NAME -y`	Roll back canary
`gco inference health NAME`	Health check
`gco inference models NAME`	List loaded models
`gco inference stop NAME -y`	Scale to zero
`gco inference start NAME`	Resume
`gco inference delete NAME -y`	Remove everything
`gco models upload PATH -n NAME`	Upload weights

Command	Description
`gco models list`	List models
`gco models uri NAME`	Get S3 URI
`gco models delete NAME -y`	Delete weights